

AI Agents

From single-shot answers to autonomous, multi-step systems — agent basics, planning, memory, tool use, multi-agent systems, agent loops, and popular agent frameworks.

DIFFICULTY LEVEL

Intermediate-Advanced

ESTIMATED READING TIME

60-75 minutes

ESTIMATED PRACTICE TIME

110-140 minutes

PREREQUISITES

Modules 01-07

LEARNING OUTCOMES

By the end of this module, you will be able to: explain what makes a system an "agent" rather than a simple chatbot; design planning and memory strategies for multi-step tasks; combine tool use into a full agent loop; understand when and how multi-agent systems help; and identify which agent framework fits a given project.

Table of Contents

1.	Introduction	
2.	Before You Begin	
3.	Main Lesson	
3.1	Agent Basics	
3.2	Planning	
3.3	Memory	
3.4	Tool Use	
3.5	Multi-Agent Systems	
3.6	Agent Loops	
3.7	Agent Frameworks	
4.	Visual Learning	
5.	Real World Applications	
6.	Practical Examples	
7.	Code Examples	
8.	Common Mistakes	
9.	Best Practices	
10.	Hands-on Activities	
11.	Mini Project	
12.	Review Questions	
13.	Cheat Sheet	
14.	Key Takeaways	
15.	Additional Resources	

1. Introduction

📖 What is this topic?

This module ties together nearly everything from Modules 01-07 — prompting, structured output, context engineering, tool use, and RAG — into a single system that can plan, act, observe, and continue autonomously across multiple steps: an AI agent.

🎯 Why is it important?

A plain chatbot answers one message at a time. An agent can be given a goal ("research and summarize competitor pricing") and independently work through the many steps needed to accomplish it — searching, reading, calculating, deciding what to do next — with minimal human intervention along the way.



🧠 Why should AI Engineers learn it?

Agents are the natural convergence point of everything else in this course: prompting techniques (Module 03, especially ReAct), context engineering (Module 05), tool use (Module 06), and RAG (Module 07) all combine directly inside agent design.

🌍 Where is it used in the real world?

Claude Code autonomously reading, editing, and testing code across a whole task; research agents that search, synthesize, and cite sources; and customer service agents that look up accounts, take actions, and escalate when needed are all real-world agents.

🔔 Why is it relevant today?

2025-2026 has seen a major industry shift from "chatbots" toward "agents" as the dominant AI product pattern, alongside the growth of standardized agent frameworks and protocols like MCP (Module 06) that make building reliable agents more practical than ever.

i Note

Agents are not a single technique but a system design pattern that combines several things you've already learned. If any concept below feels unfamiliar, it likely traces back to an earlier module worth revisiting.

2. Before You Begin

RECAP

ReAct (Module 03, 3.7) — the Thought → Action → Observation loop is the core reasoning pattern most agents run on.

RECAP

Tool use (Module 06) — agents act primarily by calling tools; without tool use, an "agent" can't actually do anything beyond generate text.



Warning

Autonomous agents that take real-world actions in a loop can compound mistakes quickly if unchecked. Always build in limits (max steps, cost caps) and human checkpoints for consequential actions.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

3.1 Agent Basics

DEFINITION

An AI agent is a system where an LLM autonomously decides a sequence of actions (often involving tool calls) to accomplish a goal, rather than producing one fixed response to one fixed input.

SIMPLE EXPLANATION

What separates an agent from a chatbot is autonomy over multiple steps: the agent itself decides what to do next based on what's happened so far, rather than a human manually driving each individual step.

 *Like the difference between a vending machine (one input, one fixed output) and hiring a personal assistant (give them a goal, and they figure out and execute the necessary steps themselves).*

EVERYDAY EXAMPLE

Instead of you manually asking "search for X," then "now summarize that," then "now compare it to Y," an agent given the goal "compare X and Y" decides and executes all of those sub-steps itself.

WHY IT MATTERS

Agents unlock tasks too complex or too multi-step for a single prompt-response cycle, while still being built from the same core building blocks covered earlier in this course.

COMMON MISTAKES

Calling any tool-using LLM call an "agent" — a single tool call with a fixed follow-up isn't autonomous multi-step decision-making, it's just tool use (Module 06).

BEST PRACTICES

Reserve "agent" architecture for tasks that genuinely need multiple, model-decided steps — simpler tasks are often better served by a straightforward tool call or prompt chain (Module 03, 3.10).

SUMMARY

An agent autonomously decides and executes a sequence of steps toward a goal — the key distinction from a single-turn chatbot or a fixed tool call.

3.2 Planning

DEFINITION

Planning is the agent's process of breaking a high-level goal into a sequence of concrete, executable steps before (or while) acting on them.

SIMPLE EXPLANATION

Some agents plan the whole sequence upfront ("first search, then extract data, then compile a report"), while others plan incrementally, deciding only the next step based on what's happened so far — connected closely to ReAct (Module 03, 3.7) and Tree of Thoughts (Module 03, 3.8).

 *Like a project manager breaking "launch the product" into a task list versus a firefighter reacting to whatever the situation demands moment to moment — both are valid planning styles depending on how predictable the task is.*

EVERYDAY EXAMPLE

A research agent given "write a report on renewable energy trends" might plan: 1) search for recent data, 2) identify 3 key trends, 3) find supporting statistics for each, 4) draft the report.

WHY IT MATTERS

Good planning prevents agents from wandering aimlessly or repeating steps — it's the difference between an agent that reaches its goal efficiently and one that loops or gets stuck.

COMMON MISTAKES

Over-committing to a rigid upfront plan for tasks where new information should change the approach mid-way (better suited to incremental, ReAct-style planning).

BEST PRACTICES

Use upfront planning for predictable, well-understood tasks; use incremental (ReAct-style) planning for tasks where the right next step depends heavily on what's discovered along the way.

SUMMARY

Planning turns a vague goal into a concrete sequence of steps — done either fully upfront or incrementally, depending on how predictable the task is.

3.3 Memory

DEFINITION

In an agent context, memory is how the agent tracks what it has already done, learned, or decided across the many steps of a single task (and sometimes across separate tasks).

SIMPLE EXPLANATION

This builds directly on Module 05's memory and conversation history concepts, but at agent scale: an agent running 20 steps needs to remember earlier tool results, decisions, and dead ends, or it will repeat work or contradict itself.

 *Like a detective's case notebook — tracking clues already checked, dead ends already ruled out, and the current leading theory, so they don't waste time re-investigating the same dead end twice.*

EVERYDAY EXAMPLE

An agent researching a topic keeps a running list of "sources already checked" so it doesn't re-search the same query repeatedly across different steps.

WHY IT MATTERS

Without effective memory, long-running agents waste steps (and cost) repeating work, and can even contradict earlier decisions within the same task.

COMMON MISTAKES

Relying purely on raw conversation history for memory in a long agent run, letting it consume the entire context window (Module 05, 3.5) without any compression or structured tracking.

BEST PRACTICES

Maintain a structured "scratchpad" (e.g., a running list of completed steps, key findings) separate from raw history, and compress or summarize as the task grows long.

SUMMARY

Agent memory tracks progress and findings across many steps — essential for avoiding repeated work and maintaining consistency in long-running tasks.

3.4 Tool Use

DEFINITION

Tool use, in an agent, is how the agent actually takes action in the world — searching, calculating, calling APIs, editing files — using the mechanism from Module 06.

SIMPLE EXPLANATION

An agent without tools can only reason and produce text; tools are what let it actually affect or query the outside world, making the difference between "planning a research report" and "actually being able to search for the information it needs."

💡 *Like the difference between a strategist who can only draw up battle plans on paper versus one who also has troops, radios, and supplies to actually execute the plan.*

EVERYDAY EXAMPLE

A coding agent's tools might include: read file, write file, run tests, search codebase — the concrete actions it uses to accomplish "fix this bug."

WHY IT MATTERS

The set of tools available directly defines the agent's capability boundary — an agent can only do what its available tools allow, no matter how good its reasoning is.

COMMON MISTAKES

Giving an agent too many overlapping or poorly described tools (Module 06, 3.5), hurting tool selection reliability at exactly the moment autonomy makes mistakes more costly.

BEST PRACTICES

Give agents the minimal, well-described tool set needed for their task domain — add tools deliberately, not "just in case."

SUMMARY

Tool use is how an agent acts — reusing everything from Module 06, now embedded inside a repeated, autonomous decision loop.

👥 3.5 Multi-Agent Systems

DEFINITION

A multi-agent system uses multiple specialized agents, each with a narrower role, coordinating together to accomplish a larger task — rather than one single generalist agent doing everything.

SIMPLE EXPLANATION

Instead of one agent trying to plan, research, write, and fact-check all at once, you might have a "researcher" agent, a "writer" agent, and a "fact-checker" agent, each focused and often better at their specific narrow task — coordinated by a controlling process or "orchestrator" agent.



💡 *Like a newsroom — a reporter gathers facts, an editor shapes the story, and a fact-checker verifies claims, each specialist doing their part better than one person trying to do all three roles simultaneously.*

EVERYDAY EXAMPLE

A coding assistant system with a "planner" agent that breaks down a feature request, and separate "implementer" and "tester" agents that each handle their specialized part.

WHY IT MATTERS

Specialized agents with narrower roles are often more reliable and easier to debug than

one generalist agent trying to do everything, similar to how focused prompt chaining (Module 03, 3.10) often beats one giant prompt.

COMMON MISTAKES

Adding multi-agent complexity for tasks simple enough for a single agent — coordination overhead isn't free, and more agents means more places for things to go wrong.

BEST PRACTICES

Reach for multi-agent systems when a task has genuinely distinct specialized roles with different concerns; start with a single agent and only split when you see real evidence it's needed.

SUMMARY

Multi-agent systems divide a complex task among specialized, coordinated agents — powerful for genuinely multi-role tasks, but added complexity that isn't always worth it.

🔄 3.6 Agent Loops

DEFINITION

An agent loop is the repeated cycle of reasoning, acting, and observing that continues until the agent's goal is achieved or a stopping condition is hit.

SIMPLE EXPLANATION

This is ReAct (Module 03, 3.7) running continuously rather than just once: think → act (tool call) → observe result → think again → act again → ... until done, with explicit conditions for when to stop (goal achieved, max steps reached, or error).



💡 *Like a thermostat's control loop — check temperature, decide an action, take it, check again, repeat — except an agent's "action" involves genuine reasoning about what to do next, not a fixed rule.*

EVERYDAY EXAMPLE

A coding agent looping: read the failing test → think about the likely cause → edit the code → run the test again → observe pass/fail → repeat until tests pass or a step limit is reached.

WHY IT MATTERS

Without an explicit, bounded stopping condition, agent loops can run indefinitely (or expensively) on tasks that are stuck, going in circles, or subtly failing without realizing it.

COMMON MISTAKES

Building an agent loop with no maximum step count or cost cap, risking runaway cost or infinite loops on edge cases.

BEST PRACTICES

Always set a maximum number of loop iterations and a clear success/failure exit condition; log each step for debuggability.

SUMMARY

The agent loop is the continuously repeating think-act-observe cycle at the heart of autonomous behavior — always bounded by explicit stopping conditions in production systems.

🧰 3.7 Agent Frameworks

DEFINITION

Agent frameworks are libraries/tools that provide pre-built infrastructure for agent loops, tool integration, memory, and multi-agent coordination, so you don't have to build every piece from scratch.

SIMPLE EXPLANATION

Popular options include LangGraph (graph-based agent/workflow orchestration), CrewAI (role-based multi-agent coordination), the OpenAI Agents SDK, and PydanticAI (type-safe agent building in Python) — each with different philosophies on how much structure vs. flexibility to provide.

Framework	Known For
LangGraph	Graph-based control flow for complex, stateful agent workflows
CrewAI	Role-based multi-agent coordination (Section 3.5) with minimal setup
OpenAI Agents SDK	Lightweight, official SDK-native agent orchestration
PydanticAI	Type-safe agent building, leaning on Pydantic (Module 04, 3.7)

💡 Like choosing a web framework (Django vs. Flask vs. FastAPI) — they all let you build a web app, but differ in how much structure they impose versus how much freedom they leave you.

EVERYDAY EXAMPLE

Using LangGraph to define an agent's steps as nodes in a graph with explicit transitions, versus writing your own bare Python loop calling the API directly (as in Section 7's code examples).

WHY IT MATTERS

Frameworks can save significant development time on common agent patterns (loops, memory, multi-agent coordination), but add a dependency and learning curve — knowing when you actually need one is a real engineering decision.

COMMON MISTAKES

Reaching for a heavyweight framework before understanding the underlying agent loop concepts (Section 3.6) — the framework becomes a black box instead of a helpful abstraction.

BEST PRACTICES

Build a simple agent loop by hand first (Section 7) to understand the mechanics, then evaluate frameworks based on your specific needs (multi-agent coordination, complex state, type safety) rather than popularity alone.

SUMMARY

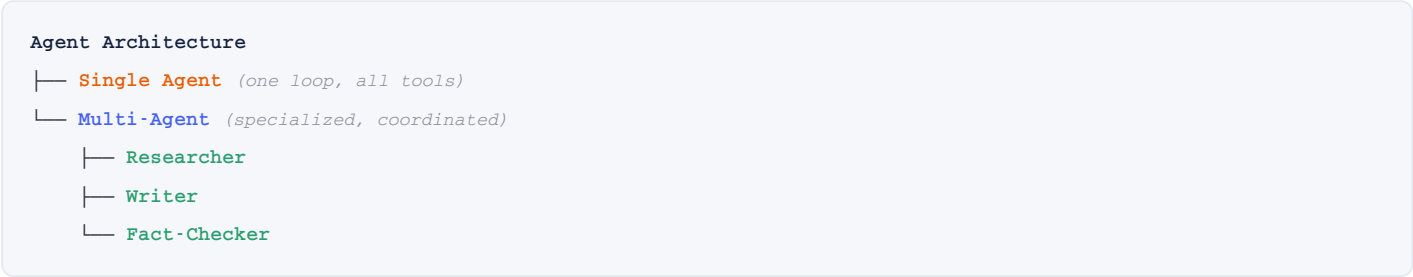
Agent frameworks provide reusable infrastructure for common agent patterns — valuable once you understand the fundamentals well enough to choose the right one deliberately.

4. Visual Learning

🔄The Agent Loop in Detail

```
| GOAL: "Research and summarize competitor pricing" |  
|  
| Step 1: Think → "I need to search for pricing data" |  
| Step 1: Act   → search_web("competitor pricing") |  
| Step 1: Observe→ [search results] |  
|  
| Step 2: Think → "I should extract key numbers" |  
| Step 2: Act   → extract_data(results) |  
| Step 2: Observe→ [structured pricing data] |  
|  
| Step 3: Think → "I have enough, time to summarize" |  
| Step 3: Act   → generate_summary(data) |  
| Step 3: Observe→ [final summary] → DONE |
```

👤Single Agent vs. Multi-Agent



🏗️Building Blocks Recap

Agent Component	Builds On
Reasoning loop	Module 03 — ReAct
Acting	Module 06 — Tool Use
Context/memory management	Module 05 — Context Engineering
Knowledge grounding	Module 07 — RAG

5. Real World Applications

Product	How these fundamentals show up
Claude Code	Runs an agent loop: reads code, plans changes, edits files, runs tests, observes results, repeats until the task is complete.
Deep research tools	Multi-step agents that search, read, synthesize, and cite across many sources autonomously.
Customer service AI	Agents that look up account data, take limited actions (refunds within policy), and escalate to humans when outside their authority.
Multi-agent content pipelines	Separate research, writing, and editing agents coordinate to produce a finished piece of content.

6. Practical Examples

Beginner: Chatbot vs. agent

Write down 3 tasks you'd give a plain chatbot (single-turn) and 3 you'd give an agent (multi-step, autonomous). Explain what makes each one fit its category.

Beginner: Manual agent loop

Pick a simple multi-step goal (e.g., "find the cheapest flight option"). Manually write out a Think → Act → Observe trace for 3 steps, as if you were the agent.

Intermediate: Designing a stopping condition

For a research agent, design 3 different stopping conditions it could use (e.g., max steps, confidence threshold, explicit "done" signal) and discuss the tradeoffs of each.

Advanced: Single vs. multi-agent design

Take a complex task (e.g., "build and test a small feature") and design both a single-agent and a multi-agent version. Compare which specialized roles would help vs. add unnecessary overhead.

7. Code Examples

🐍Python — A minimal hand-built agent loop

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

tools = [{
    "name": "search_web",
    "description": "Search the web for a query and return top results.",
    "input_schema": {"type": "object", "properties": {"query": {"type": "string"}}, "required": ["query"]}
}]

def search_web(query):
    return f"[mock search results for: {query}]" # replace with a real search API

MAX_STEPS = 6
messages = [{"role": "user", "content": "Research the pros and cons of solar panels and summarize them."}]

for step in range(MAX_STEPS):
    response = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=500, tools=tools, messages=messages
    )
    messages.append({"role": "assistant", "content": response.content})

    tool_calls = [b for b in response.content if b.type == "tool_use"]
    if not tool_calls:
        print("Agent finished:", response.content[0].text) # no more tools needed = done
        break

    # Execute each requested tool call and feed results back
    tool_results = []
    for call in tool_calls:
        if call.name == "search_web":
            result = search_web(**call.input)
            tool_results.append({"type": "tool_result", "tool_use_id": call.id, "content": result})
    messages.append({"role": "user", "content": tool_results})
else:
    print("Stopped: reached max steps without finishing.") # bounded loop, Section 3.6
```

🌐JavaScript — Same loop pattern, Node.js

```
// npm install @anthropic-ai/sdk
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

const tools = [{
    name: "search_web",
    description: "Search the web for a query and return top results.",
    input_schema: { type: "object", properties: { query: { type: "string" } }, required: ["query"] },
}];

function searchWeb({ query }) {
    return `[mock search results for: ${query}]`; // replace with a real search API
}

const MAX_STEPS = 6;
let messages = [{ role: "user", content: "Research the pros and cons of solar panels and summarize them." }];

for (let step = 0; step < MAX_STEPS; step++) {
    const response = await client.messages.create({
        model: "claude-sonnet-4-6", max_tokens: 500, tools, messages,
    });
    messages.push({ role: "assistant", content: response.content });

    const toolCalls = response.content.filter((b) => b.type === "tool_use");
    if (toolCalls.length === 0) {
        console.log("Agent finished:", response.content[0].text);
        break;
    }

    const toolResults = toolCalls.map((call) => ({
```

```
    type: "tool_result",
    tool_use_id: call.id,
    content: searchWeb(call.input),
  }));
messages.push({ role: "user", content: toolResults });
}
```



Warning

Notice the `MAX_STEPS` bound in both examples — every production agent loop needs an explicit ceiling to prevent runaway cost or infinite loops on stuck tasks.

8. Common Mistakes

- **Calling any single tool call an "agent"** — true agents involve autonomous multi-step decision-making, not one fixed call.
- **No maximum step count** on agent loops, risking runaway cost or infinite loops.
- **Relying purely on raw conversation history** for agent memory on long tasks, overflowing context.
- **Over-engineering multi-agent systems** for tasks a single agent could handle just fine.
- **Giving agents too many overlapping tools**, hurting tool selection reliability.
- **Reaching for a heavy framework** before understanding the underlying loop mechanics.

9. Best Practices

- **Reserve agent architecture** for tasks that genuinely need multiple, model-decided steps.
- **Always bound agent loops** with a max step count and clear exit conditions.
- **Maintain a structured scratchpad** for progress/findings, separate from raw conversation history.
- **Give agents a minimal, well-described tool set** specific to their task domain.
- **Start with a single agent**; split into multi-agent only with clear evidence it's needed.
- **Build a simple loop by hand first** before adopting a framework, to understand the fundamentals.

10. Hands-on Activities

Easy 5 exercises

1. Write down the difference between a chatbot and an agent in your own words, with one example of each.
2. Sketch the agent loop diagram (Section 4) from memory, then compare against the original.
3. List 3 tools a "trip planning agent" would realistically need.
4. Design 2 different stopping conditions for a simple agent task of your choice.
5. Research one agent framework (LangGraph, CrewAI, OpenAI Agents SDK, or PydanticAI) and summarize its core philosophy in 3-4 sentences.

Intermediate 3 exercises

1. Implement the minimal agent loop from Section 7 and run it on a real multi-step task with a mock tool.
2. Add a second tool to your agent and observe how it affects tool selection and step count.
3. Add a structured scratchpad (a simple list tracking completed steps) to your agent loop and print it after each step.

Challenge 2 exercises

1. Design (on paper) a 3-agent system for a content pipeline (researcher, writer, fact-checker), including what each agent's tools and inputs/outputs would be.
2. Take your Section 7 agent and deliberately create a scenario where it could loop indefinitely without a max-step bound; then add safeguards and document what you changed.

11. Mini Project

Goal

Build a small research agent that, given a topic, searches (mocked or real), extracts key points, and produces a short summary — running autonomously through multiple steps with a bounded loop.

Requirements

- An Anthropic API key
- Python or JavaScript with the Anthropic SDK
- A mock or real search tool

Step-by-Step Instructions

1. Define at least 2 tools: a search tool and a "save_finding" tool that appends to a running list of findings (your scratchpad).
2. Build the agent loop from Section 7, with a MAX_STEPS bound (e.g., 8).
3. Give the agent a research goal and let it run autonomously, calling tools as needed.
4. Print the scratchpad contents after each step to observe how findings accumulate.
5. When the agent stops calling tools, print its final summary as the result.

Expected Output

```
Step 1: Think → search for "solar panel pros and cons"
Step 1: Act   → search_web(...)
Step 1: Observe → [results]

Step 2: Think → save key finding
Step 2: Act   → save_finding("Pro: reduces electricity bills")
Scratchpad: ["Pro: reduces electricity bills"]

... (continues) ...

Step N: Agent finished — final summary printed.
```

Possible Improvements

- Add a real search API instead of a mock tool.
- Add a second specialized agent (e.g., a "fact-checker") that reviews the findings before the final summary.
- Try rebuilding the same agent using a framework (LangGraph, PydanticAI) and compare the development experience.

12. Review Questions

Multiple Choice (10)

1. What primarily distinguishes an agent from a simple chatbot?

- A) Higher temperature B) Autonomous multi-step decision-making toward a goal C) Larger context window D) Better grammar

2. Planning in an agent context means:

- A) Setting temperature B) Breaking a goal into concrete, executable steps C) Choosing a font D) Writing the system prompt only

3. Why does agent memory matter for long-running tasks?

- A) It doesn't matter B) Without it, agents may repeat work or contradict earlier decisions C) It reduces token cost to zero D) It replaces tool use

4. Tool use, in an agent, is how the agent:

- A) Thinks only B) Actually acts on/queries the outside world C) Sets its own temperature D) Chooses a persona

5. Multi-agent systems are most useful when:

- A) Every task, always B) A task has genuinely distinct specialized roles C) Never, single agents are always better D) Cost needs to be minimized at all costs

6. The agent loop is best described as:

- A) A one-time API call B) A repeated think-act-observe cycle until a stopping condition C) A type of embedding D) A prompt template

7. Why must agent loops have a maximum step count?

- A) It's optional B) To prevent runaway cost or infinite loops C) It increases creativity D) It's required by law

8. Agent frameworks like LangGraph or CrewAI primarily provide:

- A) A new LLM B) Pre-built infrastructure for agent loops, tools, and coordination C) Free API credits D) A replacement for prompting

9. What's a common mistake with agent tool sets?

- A) Too few tools always B) Too many overlapping, poorly described tools C) Using any tools at all D) Naming tools too clearly

10. It's recommended to build a simple agent loop by hand before adopting a framework because:

- A) Frameworks never work B) It builds understanding of the underlying mechanics first C) Frameworks are always slower D) It's required by every framework's license

Identification (5)

1. _____ is a system where an LLM autonomously decides a sequence of actions to accomplish a goal.

2. _____ breaks a high-level goal into concrete, executable steps.

3. _____ uses multiple specialized, coordinated agents instead of one generalist agent.

4. _____ is the repeated think-act-observe cycle at the heart of autonomous agent behavior.

5. _____ are libraries providing pre-built infrastructure for agent loops, tools, and coordination.

True or False (5)

1. Any single tool call by an LLM qualifies as a full "agent."

2. Agent loops should always have a maximum step count in production.

3. Multi-agent systems are always better than single-agent systems, regardless of task complexity.

4. Agent memory is unnecessary for long-running, multi-step tasks.

5. It's generally wise to understand basic agent loop mechanics before adopting a heavyweight framework.

Situational (5)

Situational (5)

- 1. You built an agent that occasionally runs forever on stuck tasks, racking up API costs. What's the most direct fix?**
- 2. Your agent keeps re-searching the same query multiple times within one task. What's missing from its design?**
- 3. You're building a system that needs a researcher, a writer, and an editor, each with very different concerns. What architecture fits best?**
- 4. Your agent has 15 overlapping tools and frequently calls the wrong one. What should you do?**
- 5. You're not sure if a task needs a full agent or just a single tool call. What question would help you decide?**

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-Agent, 2-Planning, 3-Multi-agent system, 4-Agent loop, 5-Agent frameworks

True or False: 1-False, 2-True, 3-False, 4-False, 5-True

Situational (sample reasoning):

1. Add a maximum step count (and possibly a cost cap) to the agent loop.
2. A structured memory/scratchpad tracking what's already been searched.
3. A multi-agent system with specialized researcher, writer, and editor agents.
4. Reduce and clarify the tool set — merge overlapping tools or write more distinct descriptions (Module 06, 3.5).
5. Does the task need multiple, model-decided steps that depend on intermediate results? If yes, agent; if it's one fixed action, a single tool call is enough.

13. Cheat Sheet

Agent vs. Chatbot

Chatbot: one turn, fixed response.

Agent: autonomous, multi-step, goal-driven.

The Loop

Think → Act → Observe → repeat, always with a max-step bound.

Multi-Agent

Use for genuinely distinct specialized roles; start single, split only with evidence.

Frameworks

LangGraph (graphs), CrewAI (roles), OpenAI Agents SDK, PydanticAI (types). Learn the loop by hand first.

14. Key Takeaways

- An agent autonomously decides and executes a sequence of steps toward a goal — the key difference from a single-turn chatbot.
- Planning breaks a goal into steps, either upfront or incrementally, depending on task predictability.
- Agent memory (a structured scratchpad) prevents repeated work and inconsistency across long-running tasks.
- Tool use defines an agent's real capability boundary — it can only do what its tools allow.
- Multi-agent systems help for genuinely distinct specialized roles, but add coordination overhead.
- Every agent loop needs an explicit, bounded stopping condition to prevent runaway cost or infinite loops.
- Agent frameworks (LangGraph, CrewAI, OpenAI Agents SDK, PydanticAI) provide reusable infrastructure — most valuable once you understand the underlying loop mechanics.

15. Additional Resources

Official Documentation

- Anthropic — Building effective agents (docs.claude.com / Anthropic engineering blog)
- LangGraph official documentation
- CrewAI official documentation
- PydanticAI official documentation

Related Modules

- Module 03 — Prompting Techniques (ReAct, the core loop pattern)
- Module 06 — Function Calling / Tool Use (how agents act)
- Module 05 — Context Engineering (agent memory and context management)

Communities & Further Learning

- Anthropic's engineering blog for real-world agent design case studies and lessons learned.